Infrastructure Kubernetes K3s

Projet DevOps B3

DevOps Team

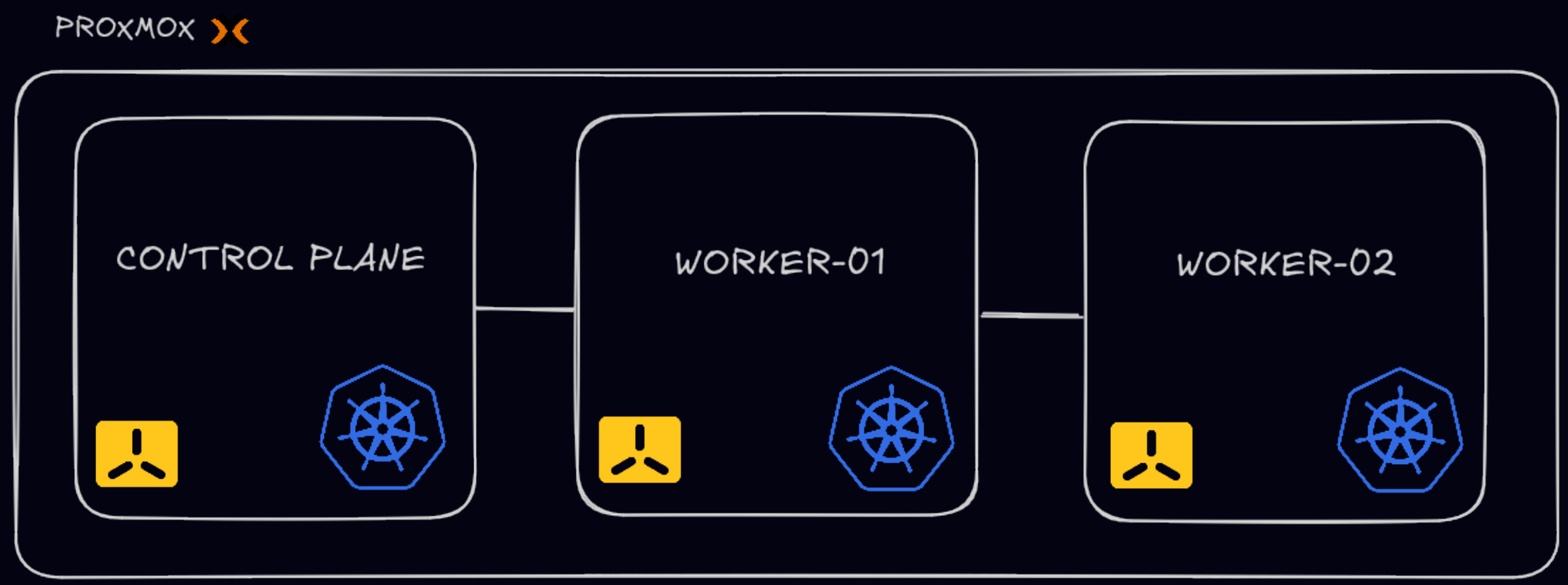
Infrastructure Kubernetes K3s

- Architecture du Cluster
- IaC / Provisionning
- Arborescence du Projet
- GitOps / Flux de Déploiement
- Applications installées
- Gestion des Secrets

Architecture du Cluster

Le cluster repose sur K3s avec des nœuds sous Ubuntu 24.04 LTS.

Node	Rôle	Système
k3s-cp	Control-Plane	Ubuntu 24.04 LTS
k3s-wk-01	Worker	Ubuntu 24.04 LTS
k3s-wk-02	Worker	Ubuntu 24.04 LTS



IaC

- Pour le déploiement des machines du cluster nous utilisons terraform avec le provider Proxmox

```
└── terraform2
    ├── main.tf
    ├── terraform.tfvars
    └── variables.tf
```

- Variable de configuration du cluster

```
variable "pm_api_url" {
  description = "URL de l'API Proxmox"
  type        = string
  default     = "https://IP_PROXMOX:8006/api2/json"
}

variable "pm_api_token_id" {
  description = "ID du token Proxmox au format user@realm!tokenid"
  type        = string
  sensitive   = true
}
```

- Utilisation des variables

```
terraform {
  required_providers {
    proxmox = {
      source  = "telmate/proxmox"
      version = "3.0.2-rc04"
    }
  }
}
provider "proxmox" {
  pm_api_url          = var.pm_api_url
  pm_api_token_id     = var.pm_api_token_id
  pm_api_token_secret = var.pm_api_token_secret
  pm_tls_insecure     = true
}
```

- Configuration des machines du cluster

```
vms = {
  "master-k3s" = {
    name          = "vm-master"
    memory        = 10240
    cores         = 5
    network_bridge = "vmbr0"
  }
  ...
}
```

Documentation complète sur Github

Arborescence du Projet

Une structure claire pour la gestion Helm et ArgoCD :

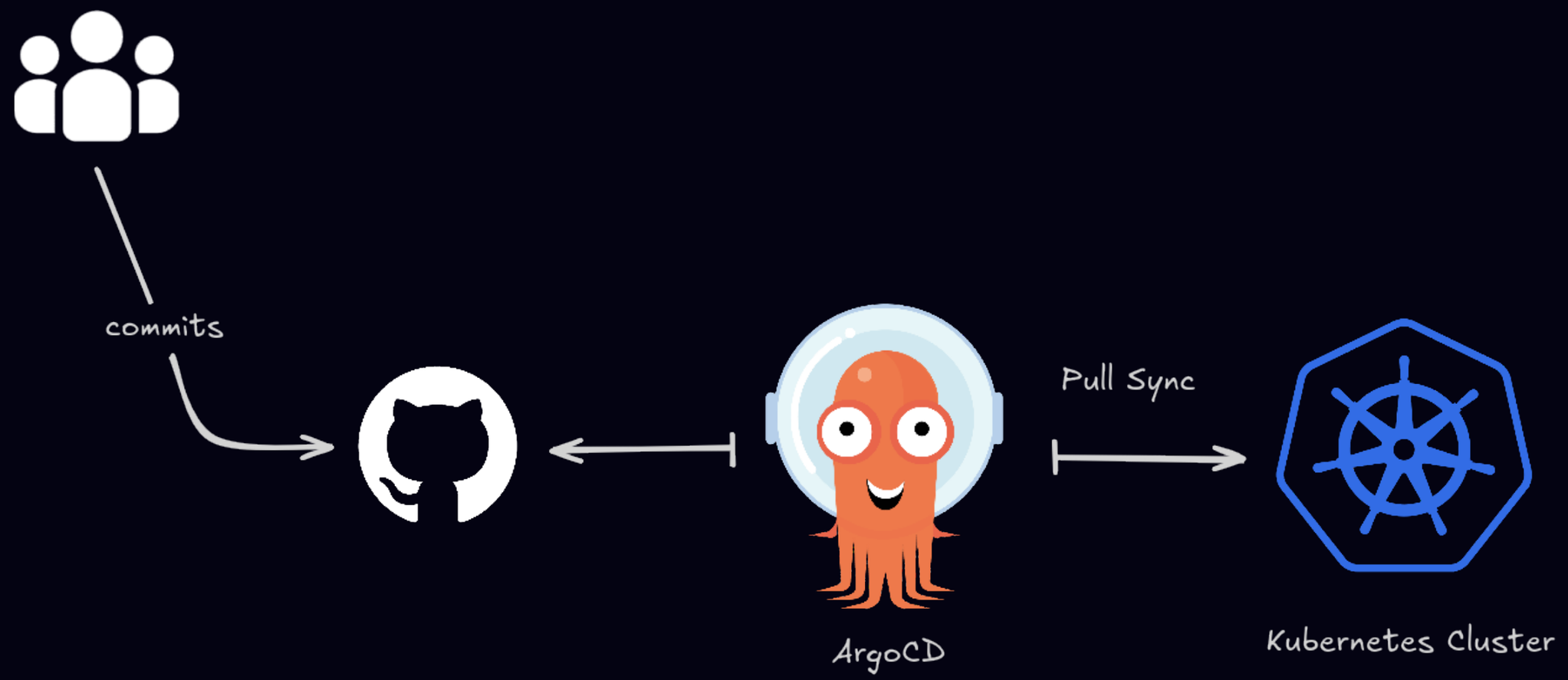
- helm-apps/ : Contient toutes les définitions d'applications.
- media/ : Assets et schémas d'architecture.
- flake.nix : Environnement de développement reproductible.

```
Devops-B3/
├── helm-apps/
│   ├── root-app.yaml         # Application "App of Apps" racine
│   ├── manifest-app/
│   │   ├── application.yaml   # ArgoCD Application
│   │   └── manifests.yaml    # Manifests Kubernetes (Deployment, Service, PVC, IngressRoute)
│   └── helm-app/
│       ├── application.yaml   # ArgoCD Application via helm chart
│       └── values.yaml       # Valeurs Helm
├── flake.nix # configuration nix
└── README.md
```

GitOps avec ArgoCD

Nous utilisons le pattern "App of Apps".

1. L'Application Racine (root-app.yaml) surveille le dossier helm-apps/.
2. Elle déploie automatiquement toute nouvelle application définie.



Flux de Déploiement

Le cycle de vie d'une modification :

1. Push sur la branche main.
2. Détection automatique par ArgoCD.
3. Synchronisation des manifests sur le cluster.
4. Vérification de l'état de santé des ressources.

Applications Déployées

Un écosystème complet de services :

- Streaming & Media : Jellyfin, Jellyseerr, Jellystat.
- Automation : n8n.
- Observabilité : Prometheus & Grafana.

```
Context: default [RW]          <0> all          <a>       Attach       <ctrl-k>  Kill            <o> Show Node
Cluster: default               <1> default      <ctrl-d>  Delete       <l>       Logs            <f> Show PortForward
User:    default               <2> kube-system  <d>       Describe     <p>       Logs Previous   <t> Transfer
K9s Rev: v0.50.18              <3> argocd       <e>       Edit         <shift-f> Port-Forward    <w> Warp To Namespace
K8s Rev: v1.34.3+k3s1                           <?>       Help         <z>       Sanitize        <y> YAML
CPU:     3%                                     <shift-j> Jump Owner   <s>       Shell
MEM:     23%
```

pods(all)[30]

NAMESPACE	NAME↑	PF	READY	STATUS	RESTARTS	CPU	%CPU/R	%CPU/L	MEM	%MEM/R	%MEM/L	IP	NODE	AGE
monitoring	alertmanager-monitoring-kube-prometheus-alertmanager-0	●	2/2	Running	6	1	n/a	n/a	60	30	n/a	10.42.2.94	k3s-wk-02	16d
argocd	argocd-application-controller-0	●	1/1	Running	1	55	n/a	n/a	405	n/a	n/a	10.42.0.55	k3s-cp	6d7h
argocd	argocd-applicationset-controller-655bc86896-bhpd7	●	1/1	Running	4	1	n/a	n/a	34	n/a	n/a	10.42.2.91	k3s-wk-02	17d
argocd	argocd-dex-server-74b7f44b64-jf9qc	●	1/1	Running	4	1	n/a	n/a	111	n/a	n/a	10.42.2.86	k3s-wk-02	17d
argocd	argocd-notifications-controller-7dc7577477-sm5mj	●	1/1	Running	4	1	n/a	n/a	109	n/a	n/a	10.42.2.85	k3s-wk-02	17d
argocd	argocd-redis-796bc79b78-vlz88	●	1/1	Running	4	4	n/a	n/a	29	n/a	n/a	10.42.0.56	k3s-cp	17d
argocd	argocd-repo-server-6748698df8-whwwg	●	1/1	Running	0	5	n/a	n/a	139	n/a	n/a	10.42.0.60	k3s-cp	4d23h
argocd	argocd-server-7c9988ccd6-whrg4	●	1/1	Running	4	3	n/a	n/a	192	n/a	n/a	10.42.1.65	k3s-wk-01	17d
kube-system	coredns-59c95cf669-t8gqw	●	1/1	Running	1	2	2	n/a	74	106	43	10.42.1.63	k3s-wk-01	6d7h
jellyfin	jellyfin-66cf54779-g4dw2	●	1/1	Running	6	1	0	0	801	78	19	10.42.1.67	k3s-wk-01	43d
jellyfin	jellyfin-exporter-78dd674677-7jpcd	●	1/1	Running	3	0	0	0	21	32	16	10.42.0.59	k3s-cp	16d
jellyfin	jellyseerr-5dd6b747d4-kdwfq	●	1/1	Running	18	19	19	1	209	81	20	10.42.2.93	k3s-wk-02	17d
jellyfin	jellystat-585b5d5784-8hwq2	●	1/1	Running	6	1	2	0	151	118	29	10.42.2.87	k3s-wk-02	17d
jellyfin	jellystat-postgres-6f59686869-dz5rq	●	1/1	Running	4	1	2	0	205	160	40	10.42.2.88	k3s-wk-02	17d
kube-system	local-path-provisioner-578895bd58-jxtsl	●	1/1	Running	6	1	n/a	n/a	46	n/a	n/a	10.42.0.54	k3s-cp	53d
metallb-system	metallb-controller-765c495b75-46znj	●	1/1	Running	6	1	n/a	n/a	72	n/a	n/a	10.42.1.61	k3s-wk-01	53d
metallb-system	metallb-speaker-mvdc9	●	4/4	Running	24	5	n/a	n/a	118	n/a	n/a	10.0.8.10	k3s-cp	53d
metallb-system	metallb-speaker-sd9nt	●	4/4	Running	24	4	n/a	n/a	119	n/a	n/a	10.0.8.20	k3s-wk-01	53d
metallb-system	metallb-speaker-xs9xv	●	4/4	Running	25	4	n/a	n/a	120	n/a	n/a	10.0.8.30	k3s-wk-02	53d
kube-system	metrics-server-7b9c9c4b9c-f4cxz	●	1/1	Running	6	3	3	n/a	75	108	n/a	10.42.0.58	k3s-cp	53d
monitoring	monitoring-grafana-67997df8c7-z2hnb	●	3/3	Running	12	9	n/a	n/a	541	n/a	n/a	10.42.2.95	k3s-wk-02	17d
monitoring	monitoring-kube-prometheus-operator-b47f48cd7-rwzl5	●	1/1	Running	4	1	n/a	n/a	75	n/a	n/a	10.42.0.57	k3s-cp	17d
monitoring	monitoring-kube-state-metrics-5b8776fc79-kxlpd	●	1/1	Running	4	1	n/a	n/a	56	n/a	n/a	10.42.2.90	k3s-wk-02	17d
monitoring	monitoring-prometheus-node-exporter-9ds64	●	1/1	Running	4	1	n/a	n/a	23	n/a	n/a	10.0.8.20	k3s-wk-01	17d
monitoring	monitoring-prometheus-node-exporter-rm6kd	●	1/1	Running	4	1	n/a	n/a	23	n/a	n/a	10.0.8.30	k3s-wk-02	17d
monitoring	monitoring-prometheus-node-exporter-s5crp	●	1/1	Running	4	2	n/a	n/a	23	n/a	n/a	10.0.8.10	k3s-cp	17d
n8n	n8n-6f9d7c589c-ws5fl	●	1/1	Running	2	2	2	0	333	130	32	10.42.2.92	k3s-wk-02	16d
monitoring	prometheus-monitoring-kube-prometheus-prometheus-0	●	2/2	Running	6	11	5	1	542	105	26	10.42.1.66	k3s-wk-01	16d
sm-operator-system	sm-operator-controller-manager-55f5c5c6d-qhj4q	●	1/1	Running	6	1	10	0	52	82	41	10.42.1.62	k3s-wk-01	51d
traefik	traefik-fbd56945c-r7jm4	●	1/1	Running	6	2	n/a	n/a	129	n/a	n/a	10.42.2.89	k3s-wk-02	53d

```
<pod>
```

Gestion des Secrets

Approche actuelle : Secrets Opaques manuels.

```
kubectl create secret generic jellystat-secrets \
  --namespace jellyfin \
  --from-literal=POSTGRES_PASSWORD='***' \
  --from-literal=JWT_SECRET='***'
```

Note : Des solutions comme HashiCorp Vault ou SOPS sont envisagées pour le futur.

Outillage : Nix

Utilisation de Nix Flakes pour garantir que tous les intervenants utilisent les mêmes versions des outils :

- kubectl
- helm
- argocd
- ...

Fini le "ça marche sur ma machine" !

Merci de votre attention !

Des questions ?

Sources

- Sources : [GitHub Repo]
- Documentation : [Kubernetes / ArgoCD / Helm]
- Tout les outils utilisés sont open-source (voir README)